Appunti di

# Architettura e progettazione dei calcolatori

Corso di laurea magistrale in

**Ingegneria Informatica**

# Indice

**1 Richiami ed approfondimenti sui sistemi di elaborazione** **5**
1.1 Richiami di calcolatori elettronici . . . . . 5
1.1.1 Architettura generale . . . . . 5
1.1.2 Istruzioni . . . . . 7
1.2 Motorola 68k . . . . . 9
1.2.1 Registri General Purpose . . . . . 9
1.2.2 Codici di Spostamento dati o indirizzi . . . . . 9
1.2.3 Codici Aritmetici . . . . . 12
1.2.4 Codici di salto . . . . . 13
1.2.5 Codici Logici . . . . . 15
1.2.6 Codici di Scorrimento . . . . . 16
1.2.7 Codici di Confronto . . . . . 16
1.2.8 Strutture sintattiche fondamentali . . . . . 17
1.2.9 Valutazione degli accessi in memoria . . . . . 19

# Capitolo 1

# Richiami ed approfondimenti sui sistemi di elaborazione

In questo capitolo sarà affrontata principalmente la parte iniziale del corso che si occupa della scrittura di programmi utilizzando il linguaggio Motorola 68k. L'interesse non sarà volto alla tipologia di architettura, anche se a volte ci sarà il bisogno di specificarla, quanto al suo utilizzo effettivo nell'ambito del corso

## 1.1 Richiami di calcolatori elettronici

Il **Motorola 68k** è un microprocessore con architettura di tipo CISC, principalmente costituita da vari registri con diverse tipologie di utilizzo. Tali registri, però, non sono una caratteristica specifica dell'architettura del Motorola 68k, pertanto è buona norma introdurre l'architettura generale di vari tipologie di microprocessori. Tali caratteristiche, quindi, non sono intrinseche del solo Motorola 68k ma sono legate alla natura stessa delle architetture dei vari processori. ANDREA GAY

### 1.1.1 Architettura generale

Quando si interagisce con le microarchitetture si lavora con vari tipologie di registri, la cui dimensione è descritta del costruttore. I registri possono essere divisi principalmente in registri utilizzabili dal programmatore (o registri utilizzabili) e quelli che non possono essere utilizzati dal programmatore (o non utilizzabili). Tale suddivisione vi è poichè alcuni registri all'interno della microarchitettura vengono utilizzati per effettuare delle operazioni pilotate dalla CU. Tali registri sono tutti interni alla CPU (Ricordando che la cpu è formata da CU, ALU e registri interni). I registri interni utilizzabili dal programmatore sono anche chiamati **registri macchina (o registri general-purpose)** e possono essere di vario tipo:

- **Registri Dato**: Registri che sono utilizzati per conservare dati su cui si può operare con varie tipologie di interazioni;
- **Registri indirizzo**: Registri che sono utilizzati per conservare gli indirizzi;
- **Registri Speciali**: Registri utilizzabili dal programmatore ma con funzioni diverse, ovvero:

- **PC (Program Counter)**: memorizza l'indirizzo in memoria della prossima istruzione;
- **IR (Istruction Register)**: Contiene una copia dell'istruzione prelevata dalla memoria;
- **SR(Status Register)**: registro di stato che contiene varie tipologie di informazione come il caso di overflow, di azzeramento del risultato, di privilegio di esecuzione (se in **administrator mode** e quindi con l'accesso ad A'7).

Tra i registri a cui invece il programmatore non ha accesso vi sono:

- **MAR (Memory Access Register)**: Registro utilizzato dal processore per scrivere l'indirizzo di memoria a cui si vuole accedere;
- **MBR (Memory Buffer Register)**: Registro che contiene il dato che si è letto/scritto in memoria (varia in base ai valori dei segnali di write e di read gestiti dalla CU);

#### 1.1.1.1 Memoria

La memoria, nell'analisi ad alto livello che stiamo conducendo, funziona come un blocco in cui sono memorizzati dati indicizzati. Per cui in base all'operazione della CU, questa modifica i valori di: MAR, MBR, segnale di read e di write. I dati possono essere acceduti in memoria in lettura/scrittura in modi diversi, e in generale la loro disposizione in memoria può variare secondo i paradigmi:

- **little-endian**: nella memorizzazione di un dato binario, riorganizzato in celle lunghe dei byte, i valori più significativi vengono memorizzati nelle celle con indirizzi più alti, mentre le meno significative in quelli con indirizzi più bassi:
- **big-endian**: nella memorizzazione di un dato binario, riorganizzato in celle lunghe 1 Byte, esso sarà memorizzato inserendo i bit più significativi nelle celle con indirizzo più basso mentre i bit meno significativi in quelle con indirizzo più alto;

Ai fini del funzionamento del processore è indifferente quale sia la tipologia di organizzazione dei dati in memoria. Ma bisogna averne una conoscenza perchè è importante far capire all'unità di controllo come deve trattare i dati che sta andando a prelevare. Nel caso del Motorola 68k ci troviamo a contatto con un processore con organizzazione big-endian. Oltre all'organizzazione dei dati un altro problema della gestione della memoria è l'**allineamento**, ovvero l'organizzazione delle celle di memoria in maniera indipendente. Nel caso del motorola 68k, sono consentiti gli accessi in memoria anche a porzioni diverse di byte, ma tali porzioni hanno il vincolo di poter essere obbligatoriamente o da 2 o da 4 Byte che iniziano in posizioni di memoria pari. Quindi si possono avere degli errori se si accede a posizioni di memoria dispari A volte quando si lavora con gli indirizzi di memoria si può andare anchein contro al problema dell'**aliasing**, ovvero un problema che riguarda l'accesso a locazioni di memoria sbagliate rispetto a quelle effettivamente desiderate (nel caso del 68k questo problema è dovuto al fatto che possiede 24 fili di bus ma i registri sono a 32 bit, per cui se prelevo un indirizzo dalla memoria, perdendo gli 8 bit più significativi, se faccio accesso all'indirizzo caricato, posso confonderlo con uno più piccolo).
La memoria quindi memorizza varie tipologie di dati e di istruzioni. Pertanto è corretto dividere la memoria in due parti principali:

- **Area codice (o area istruzioni)**: Area dove sono contenuti i programmi e le istruzioni da eseguire, in generale più piccola;
- **Area dati**: Area dove sono memorizzati i dati.

### 1.1.2 Istruzioni

In generale le istruzioni offerte dalle architetture, sono formate da tre principali parti:

- **Codici operativi**: sono la porzione di un'istruzione in linguaggio macchina che indica quale operazione la CPU deve eseguire. Insieme ai suoi operandi, è ciò che compone un'istruzione macchina. Ogni architettura ha un proprio set di codici operativi e un formato di istruzione prestabilito;
- **Operandi sorgente**: Valori che possono essere memorizzati sia in dei registri interni che esterni (in base alla tipologia di operazione), su cui poi lavorano i codici operativi;
- **Operandi destinazione**: Registri o locazioni di memoria in cui si va ad inserire il dato prodotto dai codici operativi in base agli operandi sorgente ricevuti. Solitamente tale operando è indiretto poichè potrebbe, in una successiva operazione, diventare uno dei due operandi sorgente.

In generale un istruzione è una composizione di bit conservata in memoria, in cui una parte identifica il **codice operativo** da effettuare, mentre l'altra specifica gli operandi, che nel caso di registri interni vengono identificati con il corrispettivo indirizzo, mentre nel caso di indirizzi esterni viene specificata la locazione di memoria da cui prelevare il dato. Le tipologie di indirizzamento che posso essere utilizzate per gli operandi sono:

- **Diretto**: Gli operandi presenti in memoria vengono acceduti specificando l'indirizzo di memoria in maniera plane;
- **Indiretto**: Gli operandi in memoria vengono acceduti in base al valore puntato da un registro indirizzo;
- **Implicito**: alcuni operandi sono dichiarati in maniera implicita all'interno dell'operando (Es. PUSH D3, pusha il valore in cima allo stack);
- **Immediato**: il dato da inserire in una determinata destinazione è direttamente inserito all'interno dell'istruzione (es. MOV #7,D0)
- **Spiazzamento**: la locazione di memoria a cui si vuole fare riferimento viene acceduta tramite uno spiazzamento rispetto ad un indirizzo di memoria;
- **Indice + spiazzamento**: la locazione di memoria a cui si vuole fare riferimento viene acceduta tramite un indice, che identifica una determinata zona della memoria rispetto ad un indirizzo (quindi tipo spiazzamento fisso) + un possibile ulteriore spiazzamento (per capire bene immaginarsi la memorizzazione e l'accesso a valori di una matrice);

Le istruzioni che vengono implementate per una particolare architettura vengono chiamate **ISA(Istruction Set Architecture)**, e che quindi possono essere o di tipo RISC o di tipo CISC in base alle scelte di progettazione:

- **CISC (Complex-Istruction-Set-Computer)**: Le istruzioni a disposizione del programmatore possono essere complesse, ovvero composte da più istruzioni elementari;

- **RISC (Reduced-Istruction-Set-Computer)**: L'architettura del microprocessore permette l'utilizzo di un set più ridotto di istruzioni, semplici e lineari, tali istruzioni a differenza del paradigma CISC, possono essere più veloci, ma non ripagano in termini di complessità per l'effettuazione di determinate operazioni.

Nel nostro caso noi utilizzeremo il Motorola 68k a 16/32 bit, dove tali bit indicano la grandezza dei registri, e di conseguenza, dei bus di collegamento tra essi. L'architettura di tale microprocessore è CISC, ma noi utilizzeremo un set ridotto di tutte le funzioni messe a disposizione dall'M68k, in modo da poter avere anche la confidenza giusta per affrontare, in futuro, anche tipologie di architetture RISC.
Per l'esecuzione di una particolare istruzione, il microprocessore deve, prima prelevarla dalla memoria. L'indirizzo dell' istruzione da prelevare è conservata dal Program Counter. Una volta prelevata l'istruzione dalla memoria tramite i registri MAR ed MBR, questa viene caricata nell'IR, che conserverà tale istruzione durante tutto il processo di decodifica ed esecuzione delle operazioni specificate.
Le istruzioni possono classificate in base al codice operativo nel seguente modo:

- **Trasferimento dati**: Codici che permettono di copiare un determinato valore in un registro (*MOVE*);

- **Aritmetiche**: effettua delle operazioni aritmetiche sugli operandi in ingresso e le memorizza in un operando destinazione. Solitamente le funzioni appartenenti a tale classe lavorano su numeri interi;

- **Logiche**: Operazioni che vengono effettuate sulle stringhe di bit degli operandi con una logica bit a bit. Effettuata l'operazione, il risultato viene inserito all'interno di un registro destinazione;

- **Scorrimento**: operazioni effetuate sugli operandi in ingresso che restituiscono lo scorrimento verso (destra o sinistra) dell'operando e lo memorizzano in una data locazione;

- **Confronto**: gli operandi vengono confrontati ed in base alla tipologia di controllo che voglio effettuare vado a controllare i valori dell'SR che mi interessano;

- **Salto**: Le istruzioni di salto permettono di cambiare il PC e quindi di eseguire (o rieseguire) delle porzioni di codice a cui puntano. Le istruzioni di salto possono essere di tipo condizionato (Bcc) o non condizionato (JMP). Nel primo caso l'istruzione di salto viene effettuata solo se è vera una data condizione, mentre nel secondo caso il salto viene effettuato senza il controllo di alcuna condizione;

- **Input/Output**: Alcune CPU sono dotate di apposite istruzioni per trasferire i dati da e verso le periferiche.

## 1.2 Motorola 68k

Una volta introdotti i concetti teorici e tecnologici propedeutici, si possono iniziare ad osservare i principali costrutti per la programmazione con il Motorola 68000. Conviene, quindi, non solo capire quali sono i codici per le varie tipologie di istruzioni specificate nel paragrafo [1.1.2], ma anche come costruire i principali componenti di un linguaggio di più alto livello (di cui si presuppone una minima conoscenza). I costrutti in questione possono essere: cicli, blocchi di decisione, ecc.

### 1.2.1 Registri General Purpose

Con *registri General-Purpose* (o registri macchina) intendiamo l'insieme dei registri messi a disposizione del programmatore per scrivere codice sfruttando i codici operativi dell'ISA. I registri General-Purpose a disposizione sono un insieme di registri a 32 bit:

- **Registri Dato**: Registri D0,D1,...,D7;
- **Registri Indirizzo**: Registri A0,A1,...,A7 ed A7' (questi ultimi puntano alla cima dello STACK: A7' è accessibile solo in modalità *supervisore*. In generale, ogni modalità di esecuzione dispone di un'area stack separata);
- **Status Register (SR)**: Registro a 16 bit che contiene informazioni sia sui risultati delle operazioni dell'ALU che sullo stato dell'esecuzione (se in super-user o meno). Questo regsitro è sempre accessibile in lettura, mentre è accessibile in scrittura solo in modalità supervisore.

| Bit | Sigla | Descrizione |
|---|---|---|
| 15 | T | Trace Enable |
| 14 | S | Supervisor State (1=Supervisor) |
| 13-8 | - | Non usati (riservati) |
| 7 | X | Extend (Estensione del Carry) |
| 6 | N | Negative |
| 5 | Z | Zero |
| 4 | V | Overflow |
| 3 | C | Carry |
| 2-0 | - | Non usati (riservati) |

Tabella 1.1: Bit dello Status Register (SR) del Motorola 68k

### 1.2.2 Codici di Spostamento dati o indirizzi

Il principale codice operativo che nel motorola 68k permette lo spostamento dei dati è la `MOVE`, che può essere differentemente utilizzata in base ai seguenti parametri:

- **Lunghezza degli operandi**: solitamente specificata con la *dot notation* alla fine del comando;

- **Tipologia di indirizzamento**: La `MOVE` è una tra le poche operazioni che ammette tutte le tipologie di indirizzamento possibili (proprietà detta di **ortogonalità**); L'indirizzamento immediato per l'operando di destinazione è l'unico che può causa errore, ma è un'operazione che comunque non ha alcun senso.

La caratterizzazione del comando `MOVE` è la seguente:

```
*Indirizzamento diretto D1 = D0 o D1<-D0
MOVE D0,D1

*Indirizzamento indiretto (sorgente), diretto (destinazione)
*D0 = (A0), D0 = contenuto del registro in posizione A0
MOVE.W (A0),D0

*Indirizzamento Indiretto completo
MOVE.L (A0),(A1) *(istruzione non valida)
*o
MOVEA.L (A0),A1 *(istruzione valida)
*Indirizzamento immediato + indirizzamento Diretto
MOVE.L #14,D0

* Indirizzamento con spiazzamento su registro di indirizzo
MOVE.W 4(A0), D0     * D0 = valore all'indirizzo A0 + 4

* Indirizzamento con spiazzamento e registro indice
MOVE.L 8(A0, D1.L), D2    * D2 = valore all'indirizzo A0 + 8 +
    D1

* Indirizzamento PC relativo con spiazzamento
MOVE.B 6(PC), D0    * D0 = byte situato 6 byte dopo il Program
    Counter

* Push di un registro nello Stack
MOVE.L D0, -(A7)    * Salva D0 nello stack (decremento SP)

* Pop dallo Stack in un registro
MOVE.L (A7)+, D0    * Carica D0 con il valore in cima allo
   stack (incremento SP)

* Push di un registro di indirizzo
MOVEA.L A0, -(A7)   * Salva A0 nello stack

* Pop di un registro di indirizzo
MOVEA.L (A7)+, A0   * Carica A0 con il valore in cima allo
   stack

* Salvataggio multiplo nello stack
MOVEM.L D0-D3/A0-A2, -(A7)   * Salva piu' registri nello stack

* Ripristino multiplo dallo stack
MOVEM.L (A7)+, D0-D3/A0-A2   * Ripristina piu' registri dallo
   stack
```

```

    * Ritorno da subroutine (equivalente a POP del PC)
    RTS    * Ritorna dall'ultima subroutine chiamata
```

**Nota sull'indirizzamento indiretto completo**: Non è possibile utilizzare un'operazione del tipo `MOVE (A0), (A1)` perchè il bus dati dell'architettura del MC68k non supporta simultaneamente operazioni di lettura e scrittura, dunque servono due cicli di bus distinti. Occorre utilizzare dei registri dato interni di appoggio.
**Nota su** `MOVE.L D0, -(A7)`: il pre-decremento nell'operazione di push nello stack è dovuto al fatto che, per costruzione, lo stack "cresce" verso il basso, quindi deve decrescere di 4 byte per ospitare quanto vi sto inserendo (quanto detto è scollegato dal fatto che il 68k sia big endian).
**Nota su RTS**: come concetto, RTS equivale a MOVE.L (A7)+, PC perché il valore puntato da A7 è quello * in cima allo stack, in questo caso estraiamo quindi + (lo stack "sale"), in caso contrario avremmo avuto - (quando inseriamo nello stack).
Nel codice precedente sono da notare le seguenti notazioni:

- **.W**: tale parte del comando permette di capire che sto lavorando con operandi lunghi 16 bit (o 2 Byte) esse sono denotate Word. Nel caso in cui non sia specificato alcun markup, allora la move è da intendersi per soli 8 bit
- **.L**: tale parte del comando permette di capire che sto lavorando con operandi lunghi 32 bit (o 4 Byte) esse sono denotate Long Word
- **#14**: Vado ad identificare un valore immediato tramite il termine #<valore>, che sarà convertito in binario dal compilatore e poi inserito all'interno del programma, quindi integrato all'interno della zona istruzioni della mia memoria

### 1.2.3 Codici Aritmetici

Per il motorola vi sono vari codici aritmetici, che però possono lavorare solo su valori interi e quindi non valori "reali" (o codificati in IEEE 754). I codici aritmetici più importanti, ed in generale, più presenti all'interno delle varie architetture sono i seguenti:

#### 1.2.3.1 Somma

```
*Operatore di somma
ADD #3, D0  *immediato + diretto, D0 = 3+D0
ADD.W #3,D0 *somma con specifica grandezza valore D0 = 3+D0
ADD.W D0,D1 *indirizzamenti diretti con D1 = D0+D1

ADDA.L #1,A0 *somma su registri di tipo indirizzo, somma 1
   direttamente al contenuto di A0, non alla locazione di
   memoria puntata da A0 (NON ci sonon le parentesi)

ADDQ.W #1,D0 *somma di un valore immediato tra 1 e 8
```

#### 1.2.3.2 Sottrazione

```
*Operatore di Sottrazione
SUB #3, D0  *immediato + diretto, D0=D0-3
SUB.W #3,D0 *sotterazione con specifica grandezza valore D0=
   D0-3
SUB.W D0,D1 *indirizzamenti diretti con D1=D1-D0

SUBA.L #1,A0 *Sottrazione su registri di tipo indirizzo

SUBQ.W #1,D0 *Sottrazione di un valore immediato tra 1 e 8
```

#### 1.2.3.3 Moltiplicazione

```
MULU #3, D0       * Moltiplicazione senza segno immediato +
   diretto, D0 = D0 * 3
MULU.W #3,D0      * Moltiplicazione senza segno con specifica
   grandezza valore, D0 = D0 * 3
MULU.W D0,D1      * Moltiplicazione senza segno con
   indirizzamento diretto, D1 = D1 * D0

MULS.W #3,D0      * Moltiplicazione con segno immediato +
   diretto, D0 = D0 * 3
MULS.W D0,D1      * Moltiplicazione con segno tra registri, D1
    = D1 * D0
```

#### 1.2.3.4 Divisione

```
DIVU #3, D0        * Divisione senza segno immediato + diretto, D0 = D0 / 3 (quoziente in D0, resto in D1)
DIVU.W #3,D0       * Divisione senza segno con specifica grandezza valore, D0 = D0 / 3
DIVU.W D0,D1       * Divisione senza segno con indirizzamento diretto, D1 = D1 / D0

DIVS.W #3,D0       * Divisione con segno immediato + diretto, D0 = D0 / 3
DIVS.W D0,D1       * Divisione con segno tra registri, D1 = D1 / D0
```

Come si nota dalle varie implementazioni dei codici aritmetici, questi non possono utilizzare tipologie di indirizzamento indiretto. Pertanto, prima di effettuare le operazioni aritmetiche, gli operandi di input devono essere caricati nei registri interni ed il risultato sarà poi memorizzato in uno dei due registri impiegati (Come visibile nei commenti dei vari comandi). Inoltre, sia MULU e MULS che DIVU e DIVS lavorano implicitamente su 16 bit nel 68k, e non esistono cose del tipo MULU.B oppure MULU.L (idem per gli altri 3 codici operativi).

### 1.2.4 Codici di salto

I codici di salto possono essere di 3 tipologie principali nel motorola 68k:

- **Salti incondizionati**: Quando viene incontrata l'istruzione di salto, questa effettua il salto (cambiamento del PC) in maniera immediata e senza la verifica di alcuna condizione;
- **Salti condizionati**: I salti condizionati sono effettuati in base al verificarsi di una determinata condizione. Nel caso del motorola 68k la condizione è associata ai valori dei singoli bit dello Statur Register (SR);
- **Salti a subrutine**: I salti a subroutine sono delle tipologie particolari di salto incondizionato, con l'unica differenza che l'indirizzo di memoria da cui si è saltati viene prima memorizzato nello stack e poi viene effettuato il salto. Tale operazione fa in modo che una volta eseguita la subroutine, il sistema possa ritornare al punto a cui si era fermato nel programma principale, sollevando il programmatore dall'onere di gestire manualmente il salvataggio dell'indirizzo di ritorno.

In generale, quando si definisce un codice di salto, bisogna prevedere anche il suo operando, che dal lato del programmatore può essere di principalmente 2 tipi:

- **Label**: dopo l'istruzione si fa riferimento ad una label all'interno del programma scritto che permette di evitare di andare a lavorare con indirizzi di memoria diretti (sarà il compilatore a configurarli ad-hoc);
- **Indirizzamento indiretto**: Tramite dei registri indirizzo si indica la locazione di memoria specifica a cui si vuole saltare;

#### 1.2.4.1 Salti non condizionati

I salti non condizionati in motorola possono essere i seguenti:

```
    BRA label       * Salto incondizionato alla label specificata (branch always)
    JMP address     * Salto incondizionato all'indirizzo specificato
    JMP (A0)        * Salto all'indirizzo contenuto in A0
```

Solitamente nelle varie applicazioni si preferisce utilizzare il comando BRA, poichè più semplice da ricordare in riferimento ai comandi di salto condizionato.

#### 1.2.4.2 Salti condizionati

I salti condizionati hanno una forma più o meno uguale in base a quello che si vuole fare. La loro forma nel caso del 68k è del tipo: Bcc. Dove la B sta per BRANCH mentre "cc" sono le componenti che permettono di distinguere la condizione da considerare rispetto ai valori dello SR. In particolare, i salti condizionati operano in base ai valori dei 5 bit del **CCR (Condition Code Register)**, che sono i 5 bit meno significativi dello SR riportati in tabella 1.1.
I principali comandi sono i seguenti:

```
    BCS label       * Salto se Carry e' settato (C = 1)
    BCC label       * Salto se Carry e' azzerato (C = 0)
    BVS label       * Salto se Overflow e' settato (V = 1)
    BVC label       * Salto se Overflow e' azzerato (V = 0)
    BEQ label       * Salto se Zero e' settato (Z = 1)
    BNE label       * Salto se Zero e' azzerato (Z = 0)
    BMI label       * Salto se Negativo e' settato (N = 1)
    BPL label       * Salto se Positivo e' settato (N = 0)

    BLT label       * Salto se Minore di (N XOR V = 1)
    BLE label       * Salto se Minore o uguale ((N XOR V) + Z = 1)
    BGT label       * Salto se Maggiore ((N XOR V) + Z = 0)
    BGE label       * Salto se Maggiore o uguale (N XOR V = 0)

    BLS label       * Salto se Minore o uguale (C + Z = 1) (senza segno)
    BHI label       * Salto se Maggiore (C + Z = 0) (senza segno)
```

#### 1.2.4.3 Salti a subroutine

I salti a subroutine sono una tipologia di salto incondizionato. Tali salti fanno parte di architetture CISC principalmente, poichè alcune architetture RISC non prevedono tali funzioni. La presenza di tali funzioni permette di non dover gestire l'indirizzo di ritorno dalla subroutine, o almeno non direttamente dal programmatore. Le istruzioni che in motorola 68k sono principalmente utilizzate per la chiamata a subroutine sono le seguenti:

```
    * Salta a una subroutine e salva il ritorno nello stack
    * (push nello stack)
    BSR label
    *Salta a una subroutine con indirizzo specifico o label
    JSR address/label
    * Ritorna dalla subroutine (estrae l'indirizzo di
    * ritorno dallo stack, fa il pop dallo stack)
    RTS
```

### 1.2.5 Codici Logici

I codici logici sono operazioni che possono essere effettuate su degli operandi lavorando bit a bit. Da notare che per i codici logici si può avere l'indirizzamento indiretto solo per la sorgente e non per il destinatario. Esempi di codici logici sono i seguenti:

```
    AND #3, D0        * AND bit a bit con valore immediato, D0=D0&3
    AND.W D0, D1      * AND tra registri, D1=D1&D0
    AND.L (A0), D0    * AND tra valore puntato da A0 e D0

    OR #3, D0         * OR bit a bit con valore immediato, D0=D0|3
    OR.W D0, D1       * OR tra registri, D1=D1|D0
    OR.L (A0), D0     * OR tra valore in memoria puntato da A0 e D0

    EOR #3, D0        * XOR bit a bit con valore immediato, D0=
       D0XOR3
    EOR.W D0, D1      * XOR tra registri, D1 = D1 XOR D0
    EOR.L (A0), D0    * XOR tra valore puntato da A0 e D0

    NOT D0            * Complemento bit a bit (negazione), D0 = ~D0
```

### 1.2.6 Codici di Scorrimento

I codici di scorrimento permettono di effettuare delle operazioni di shift, che possono essere comode in alcune tipologie di operazioni

```
    ASL #1, D0        * Shift aritmetico a sinistra di 1 bit
                      *(mantiene il segno)
    ASR #1, D0        * Shift aritmetico a destra di 1 bit

    LSL #1, D0        * Shift logico a sinistra di 1 bit (0 fill)
    LSR #1, D0        * Shift logico a destra di 1 bit

    ROL #1, D0        * Rotazione a sinistra di 1 bit
                      * (il bit piu' alto rientra da destra)
    ROR #1, D0        * Rotazione a destra di 1 bit

    ROXL #1, D0       * Rotazione a sinistra con Carry
    ROXR #1, D0       * Rotazione a destra con Carry
```

Osserviamo che lo shift aritmetico a sinistra ha l'effetto di moltiplicare per 2 (se convertiamo da decimale a binario il valore del registro prima e dopo lo shift), tranne se perdi bit significativi o alteri il segno, e dopo lo shift aritmetico possono essere modificati i bit del CCR.

### 1.2.7 Codici di Confronto

I codici di confronto sono molto importanti, poiché in accoppiata con i salti condizionati permettono di costruire tutti i costrutti fondamentali che possiamo trovare anche nei linguaggi di alto livello

```
    CMP #5,D0         * Confronta D0 con 5 (D0 - 5, senza
                      * modificare D0, aggiorna i flag)
    CMP.W D0,D1       * Confronta D1 con D0 (D1 - D0,
                      * aggiorna solo i flag)
    CMP.L (A0),D0     * Confronta D0 con il valore in
                      * memoria puntato da A0
    CMPI #10,D0       * Confronto immediato con D0
                      * (D0 - 10, aggiorna solo i flag)
    CMPA.L #1000,A0   * Confronta registro indirizzo A0 con 1000
```

Oltre a semplici comparazioni, solitamente, vi sono anche dei comandi che operano sui singoli registri. Non solo per il controllo di tali registri ma anche per l'effettuazione di eventuali operazioni che possono essere comode per una tipologia di interpretazione ad alto livello del codice

```
    TST D0            * Testa D0 (controlla se e' zero o negativo,
                      * senza modificarlo)
    TST.W (A0)        * Testa il valore in memoria puntato da A0
    BTST #3, D0       * Testa il bit 3 di D0
                      * (imposta Z se il bit e' 0)
    BTST #5, (A0)     * Testa il bit 5 della memoria puntata da A0
    BSET #3, D0       * Imposta il bit 3 di D0 a 1
```

```
    BSET #5, (A0)    * Imposta il bit 5 della memoria puntata da A0
    BCLR #3, D0      * Azzera il bit 3 di D0
    BCLR #5, (A0)    * Azzera il bit 5 del registro puntato da A0
    BCHG #3, D0      * Inverte il bit 3 di D0 (0 -> 1, 1 -> 0)
    BCHG #5, (A0)    * Inverte il bit 5 del registro puntato da A0
    TAS D0           * Testa e imposta il bit piu' alto (7) di D0
    TAS (A0)         * Testa e imposta il bit 7 del valore in
                     * memoria puntato da A0
```

### 1.2.8 Strutture sintattiche fondamentali

Dati i codici di **Salto** [1.2.4] e quelli di **Confronto** [1.2.7], si possono costruire quelle che sono le struttura sintattiche fondamentali

#### 1.2.8.1 if-then-else

Per costruire il ciclo if-then-else bisogna per prima cosa comprendere quale sia la condizione, poichè bisognerà identificare:

- **Registro target**: In base a quale registro/operazione devo decretare la condizione?
- **Condizione**: Qual'è la condizione da rispettare?

Scelti questi due parametri allora sarò capace di capire quale codice cmp utilizzare ed in che modo, e quale tipologia di salto condizionato effettuare. Esempio di un classico If-Then:

```
        MOVE.L  D0, D1   * Carica valori nei registri
                         * (supponiamo che D0 e D1 abbiano gia' valori)
        CMP.L   D1, D0   * Confronta D0 con D1 (D0 - D1)
        BGT     END_IF   * Se D0 > D1, salta al blocco then
                         * (condizione = (D1 <= D0))
THEN                     * Codice interno all'IF
END_IF                   * Codice successivo...
```

Esempio di un If-Then-Else

```
        MOVE.L  D0, D1   * Carica valori nei registri (supponiamo che D0
                         * e D1 abbiano gia' valori)
        CMP.L   D1, D0   * Confronta D0 con D1 (D0 - D1)
        BGT     THEN     * Se D0 > D1, salta al blocco THEN
ELSE    MOVE.L  #0, D2   * D2 = 0
        BRA     END_IF   * Salta oltre il blocco THEN
THEN    MOVE.L  #1, D2   * D2 = 1
END                      * Codice successivo...
```

#### 1.2.8.2 Ciclo FOR

Come per l'if il ciclo for è composto principalmente da codici di **salto** e da codici di **confronto**. La struttura è molto simile a quella dell'If-Then-Else con l'eccezzione della posizione dei vari salti. Precisamente la struttura di un ciclo for è la seguente:

```
    MOVE.L   #0, D0   * Inizializza il contatore D0 = 0

FOR CMP.L    #10, D0 * Confronta D0 con 10
    BGE      END      * Se D0 >= 10, esce dal ciclo

                      * Corpo del ciclo ...
    ADDQ.L   #1, D0   * Incrementa D0 di 1
    BRA      FOR      * Ripete il ciclo
END
```

#### 1.2.8.3 Ciclo While

Il ciclo while segue le regole del ciclo for solo con una condizione differente:

```
WHILE    CMP.L    #0, D1   * Confronta D1 con 0
         BLE      END      * Se D1 <= 0, esce dal ciclo
                           * Corpo del ciclo
         SUBQ.L   #1, D1   * Decrementa D1 di 1
         BRA      WHILE    * Ripete il ciclo
END                        * Codice successivo
```

#### 1.2.8.4 Chiamata a subroutine

Le chiamate a subroutine possono essere viste come una sorta di chiamate a funzione. Esse, quindi, possono avere sia degli operandi di ingresso che degli operandi di uscita. La "comunicazione" degli operandi con la subroutine può avvenire in due principali modi:

- **Con registri interni**: Gli operandi vengono caricati nei registri interni prima di chiamare la subroutine, che poi ci lavorerà sopra. Quindi i registri interni vengono utilizzati come una sorta di canale di comunicazione.
- **Con Stack**: Gli operandi sono allocati sullo stack, ciò richiede quindi una gestione anche del puntatore dello stack SP.

Un esempio di chiamata a subroutine con memorizzazione degli operandi nello stack è il seguente:

```
         MOVE.L   #5, D0       * Carica il primo operando in D0
         MOVE.L   #10, D1      * Carica il secondo operando in D1
         MOVE.L   D0, -(A7)    * Push del primo operando nello stack
         MOVE.L   D1, -(A7)    * Push del secondo operando
         JSR      SUM_SUB      * Chiamata alla subroutine
         MOVE.L   (A7)+, D2    * prelievlo del risultato dallo stack
         ADDQ.L   #8, A7       * Pulizia dello stack
                               * (2 valori da 4 byte)
                               * D2 ora contiene il risultato
```

```
                                * Codice successivo eventuale
SUM_SUB MOVE.L  (A7)+, D0       * Pop del primo operando dallo stack
        MOVE.L  (A7)+, D1       * Pop del secondo operando
        ADD.L   D1, D0          * Somma D0 + D1, risultato in D0
        MOVE.L  D0, -(A7)       * Push del risultato nello stack
        RTS                     * Ritorna al chiamante
```

Esempio di utilizzo dei registri interni, sia per passaggio operandi di ingresso che di uscita:

```
        MOVE.L  #5, D0          * Primo operando in D0
        MOVE.L  #10, D1         * Secondo operando in D1

        JSR     SUM_REGS        * Chiamata alla subroutine
        * Dopo il ritorno, il risultato e' in D0
        * Codice successivo...
SUM_REGS:   ADD.L   D1, D0      * Somma D0 + D1, risultato in D0
        RTS                     * Ritorna al chiamante
```

### 1.2.9 Valutazione degli accessi in memoria

Utilizzando i comandi precedentemente presentati, il processore può accedere in memoria principale. Gli accessi in memoria dipendono fortemente dalla tipologia di architettura che ho adottato. In generale gli accessi in memoria possono avvenire per 2 tipologie di operazioni: Accesso in memoria per le Istruzioni (PI) o accesso in memoria Per le Operazioni (PO). Veodiamo degli esempi per capire meglio di cosa si sta parlando:

| Istruzione | PI | PO |
|---|---|---|
| MOVE.L D0,D1 | 1 | 0 |
| MOVE.W D0,D1 | 1 | 0 |
| MOVE.L #7,D1 | 3 | 0 |
| MOVE.W (A0),(A1) | 1 | 2 |
| MOVE.W (A0),VAR | 3 | 2 |

Tabella 1.2: Conteggio accessi per Architettura a 16 bit e VAR a 32 bit

Per contare gli accessi in memoria bisogna effettuare delle osservazioni in base alla parte che si sta analizzando. Per l'accesso **Per le Istruzioni (PI)** si ragiona sui seguenti accessi:

- **Prelievo dell'istruzione**: Un operazione che non mancherà mai sarà sempre il prelievo dell'istruzione, che impone che il mio PI non potrà mai essere nullo;

- **Operandi immediati**: se nel mio comando ho degli operandi immediati, allora dovrò accedere anche altre volte alla memoria per il prelievo di tale operando. I miei accessi, per questo caso, sono dettati dalla lunghezza dell'operando rispetto ai miei bus disponibili. Per esempio, se sono in un architettura a 16 bit devo prelevare un immegiato considerato una WORD, allora il numero di accessi aggiuntivi per prelevare l'immediato è uguale a 1. Mentre se stessi lavorango con le Long World (32 bit), allora il numero di accessi, a parità di architettura, sarà 2;

- **Variabili**: se sto utilizzando delle variabili, che indicano delle locazioni di memoria dirette, allora dovrò fare un numero di accessi alla memoria che mi permette di prelevare gli indirizzi (tali indirizzi sono lunghi tutti 32 bit). Pertanto con un architettura a 16 bit dovrò considerare sempre 2 accessi per ogni variabile per prelevare tali indirizzi;

Per l'accesso **Per gli Operandi (PO)**, i parametri risultano più o meno gli stessi, ad esclusione del prelievo istruzione e della variabile. Le operazioni che si contano per tale processo sono:

- **Indirizzamento Indiretto**: quando vado ad effettuare dei riferimenti a dei registri della memoria con indirizzamento indiretto allora devo prevedere un numero di accessi per il prelievo dell'operando. Quindi bisogna conteggiare il numero di accessi per il prelievo dell'operando in base alla sua lunghezza. Nel caso di architettura a 16 bit si avranno: 1 accesso per prelevare delle word (16 bit) e 2 accessi pre prelevare le Long Word (32 bit)

- **Variabili**: Quando si utilizzano le variabili, oltre a prelevare gli indirizzi dalla "zona istruzioni" bisogna prelevare gli operando. La conta del numero di accessi per il prelievo degli operandi è uguale al caso di indirizzamento indiretto

Nel caso degli accessi PO, si è prevista la considerazione per singoli operandi. Quindi in base al numero di operandi presenti, la loro lunghezza prefissata, il loro modo di indirizzamento, si riesce a comprendere (cumulando le specifiche), il numero di accessi che bisogna effettuare in memoria ed il motivo di tali accessi.